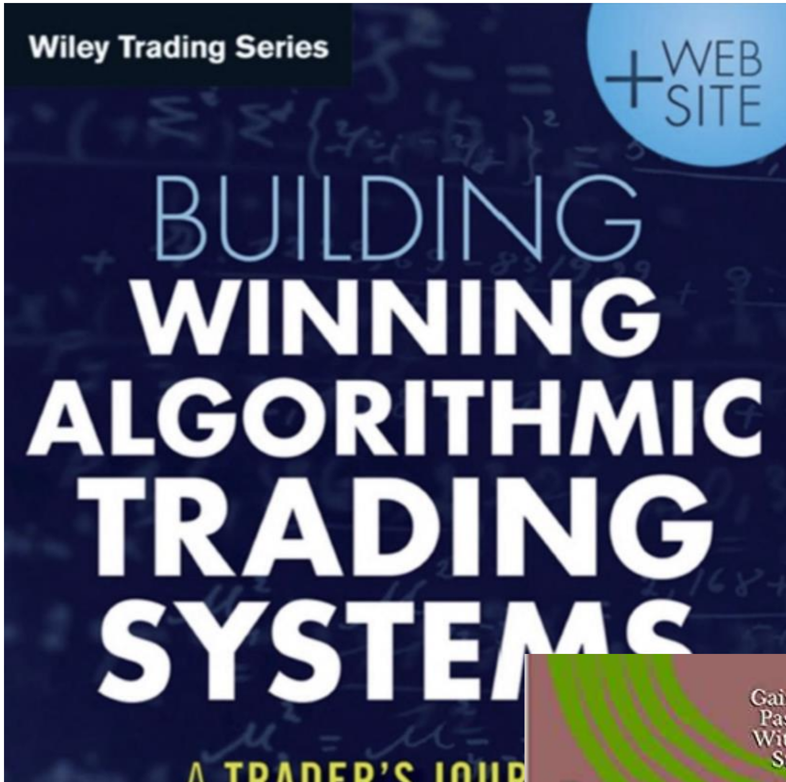
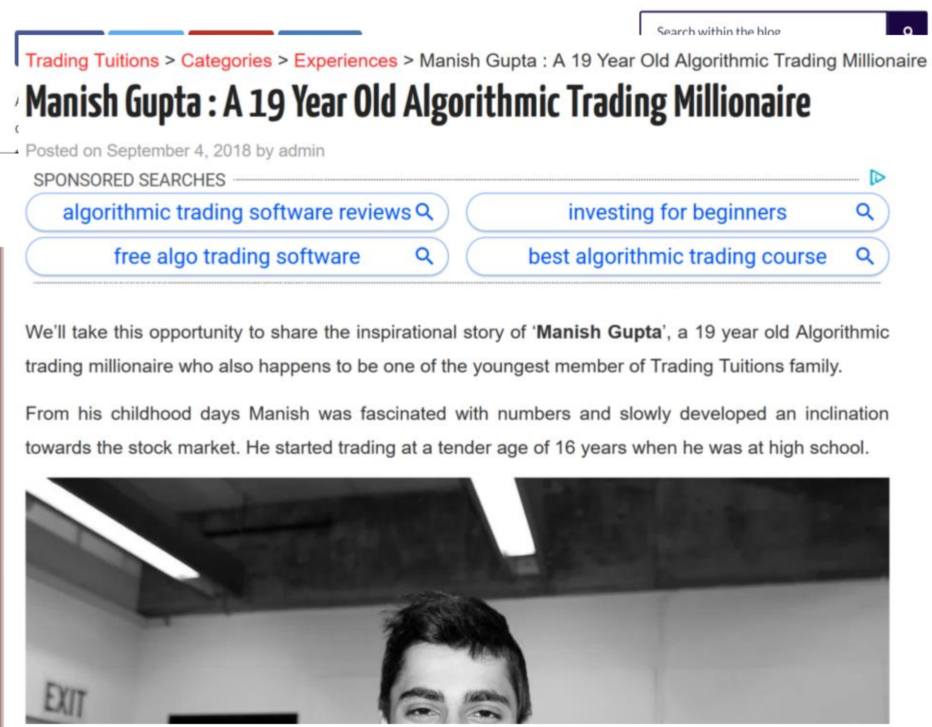
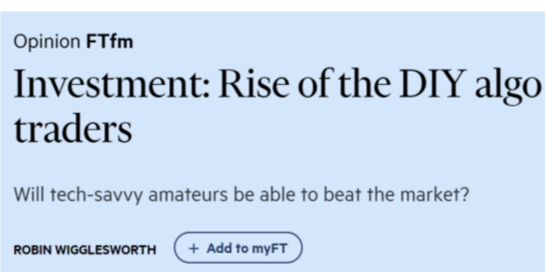
Hochschule für Technik
und Wirtschaft Berlin

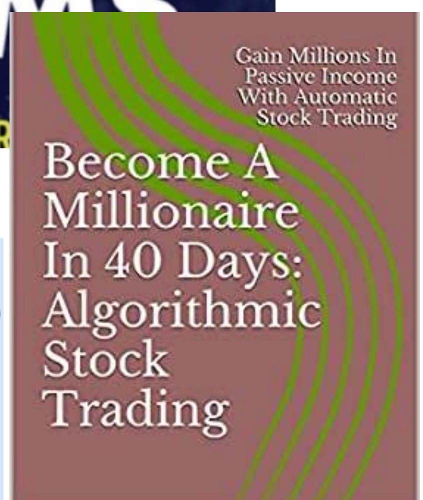
University of Applied Sciences

Algorithmic Trading









Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Research

THE JOURNAL OF FINANCE • VOL. XLVII, NO. 5 • DECEMBER 1992

Simple Technical Trading Rules and the Stochastic Properties of Stock Returns

WILLIAM BROCK, JOSEF LAKONISHOK, and
BLAKE LEBARON*

ABSTRACT

This paper tests two of the simplest and most popular trading rules—moving average and trading range break—by utilizing the Dow Jones Index from 1897 to 1986. Standard statistical analysis is extended through the use of bootstrap techniques. Overall, our results provide strong support for the technical strategies. The returns obtained from these strategies are not consistent with four popular null models: the random walk, the AR(1), the GARCH-M, and the Exponential GARCH. Buy signals consistently generate higher returns than sell signals, and further, the returns following buy signals are less volatile than returns following sell signals, and further, the returns following buy signals are less volatile than returns following sell signals. Moreover, returns following sell signals are negative, which is not easily explained by any of the currently existing equilibrium models.

ELSEVIER

Review of Financial Economics

Volume 23, Issue 1, January 2014, Pages 30-45

Predictability of the simple technical trading rules: An out-of-sample test

Jiali Fang, Ben Jacobsen, Yafeng Qin

Show more

https://doi.org/10.1016/j.rfe.2013.05.004

Abstract

In a true out-of-sample test based on fresh data we find no evidence that several well-known technical trading strategies predict stock markets over the period of 1987 to 2011. Our test safeguards against sample selection bias, data mining, hindsight bias, and other usual biases that may affect results in our field. We use the exact same technical trading rules that Brock, Lakonishok, and LeBaron (1992) showed to work best in their historical sample. Further analysis shows that this

Secretive Community

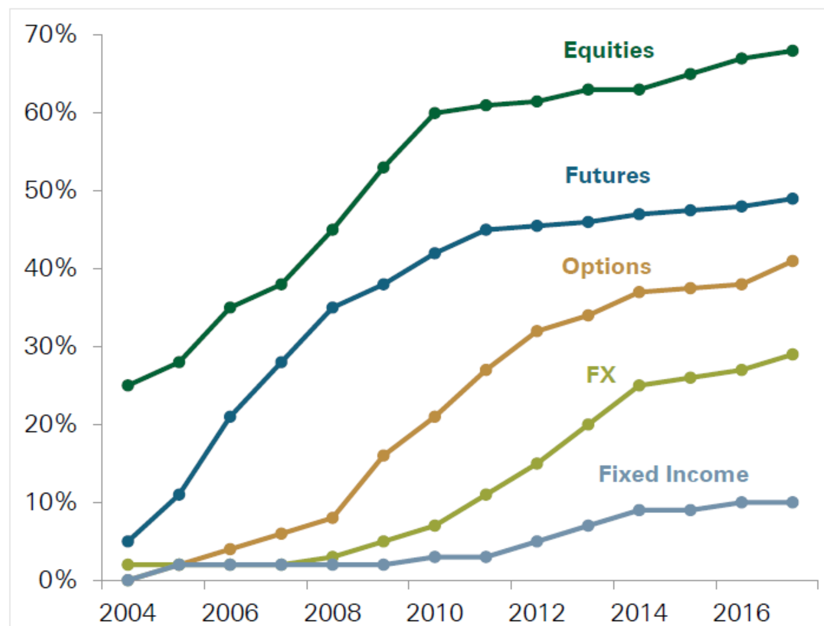
DOI:10.1145/2500117

The competitive nature of AT, the scarcity of expertise, and the vast profits potential, makes for a secretive community where implementation details are difficult to find.

BY PHILIP TRELEAVEN, MICHAL GALAS, AND VIDHI LALCHAND

Algorithmic Trading Review, Communications of the ACM, 2013

Exhibit 2: Market Share of Algorithmic Trading by Asset Class



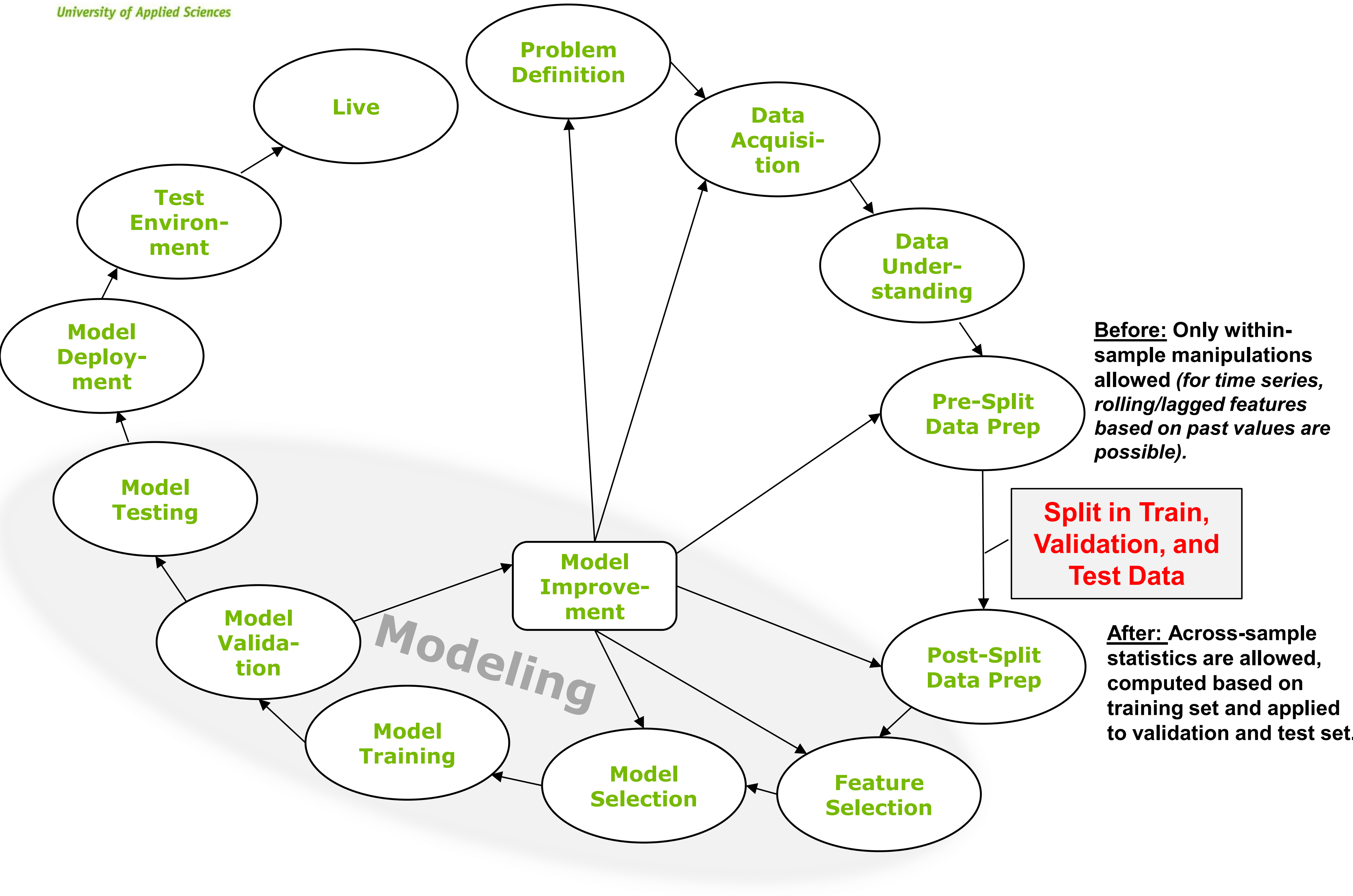
As of 2017.

Source: Goldman Sachs, AiteGroup

Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Data Science Process



Hochschule für Technik
und Wirtschaft Berlin
University of Applied Sciences

Backtesting of Various Strategies

- Targets:
  - Markets (NASDAQ, DAX, currencies, crypto, etc.)
  - Frequency (monthly, weekly, daily, hourly, every minute)
  - Individual stocks, industries, markets, indices
  - Returns based on selling strategies (e.g. signals from technical indicators, stop-loss and profit-taking strategies)
  - …
- Features:
  - Bar data (Open-High-Low-Close-Volume)
  - Trade and quote data
  - Chart patterns (TAlib)
  - Correlated series (stocks, raw materials, etc.)
  - News events
  - Signals from common trading strategies (e.g. https://enrichmoney.in/knowledge-center-chapter/intraday-trading-strategies)
  - …
- Data acquisition:
  - Yfinance
  - Alpaca
  - Alphavantage
  - …
- Modeling architecture:
  - Decision Trees, LSTM, RNN, Attention mechanism
  - Dimensionality (number of layers, number of neurons, etc.)
  - Activation functions (ReLU, Tanh, etc.)
  - Mechanisms (drop out, etc.)
  - Optimization (Adam, SGD, etc.)
  - …
- Performance criteria:
  - Return over period of time
  - Accuracy of prediction
  - Sharpe Ratio
  - …
- Baseline:
  - Market performance over period of time
  - Comparison with dummy strategy (randomly buy and sell, etc.)
  - ..

Hochschule für Technik
und Wirtschaft Berlin

Literature Review 2020 - 2024

Contents lists available at ScienceDirect

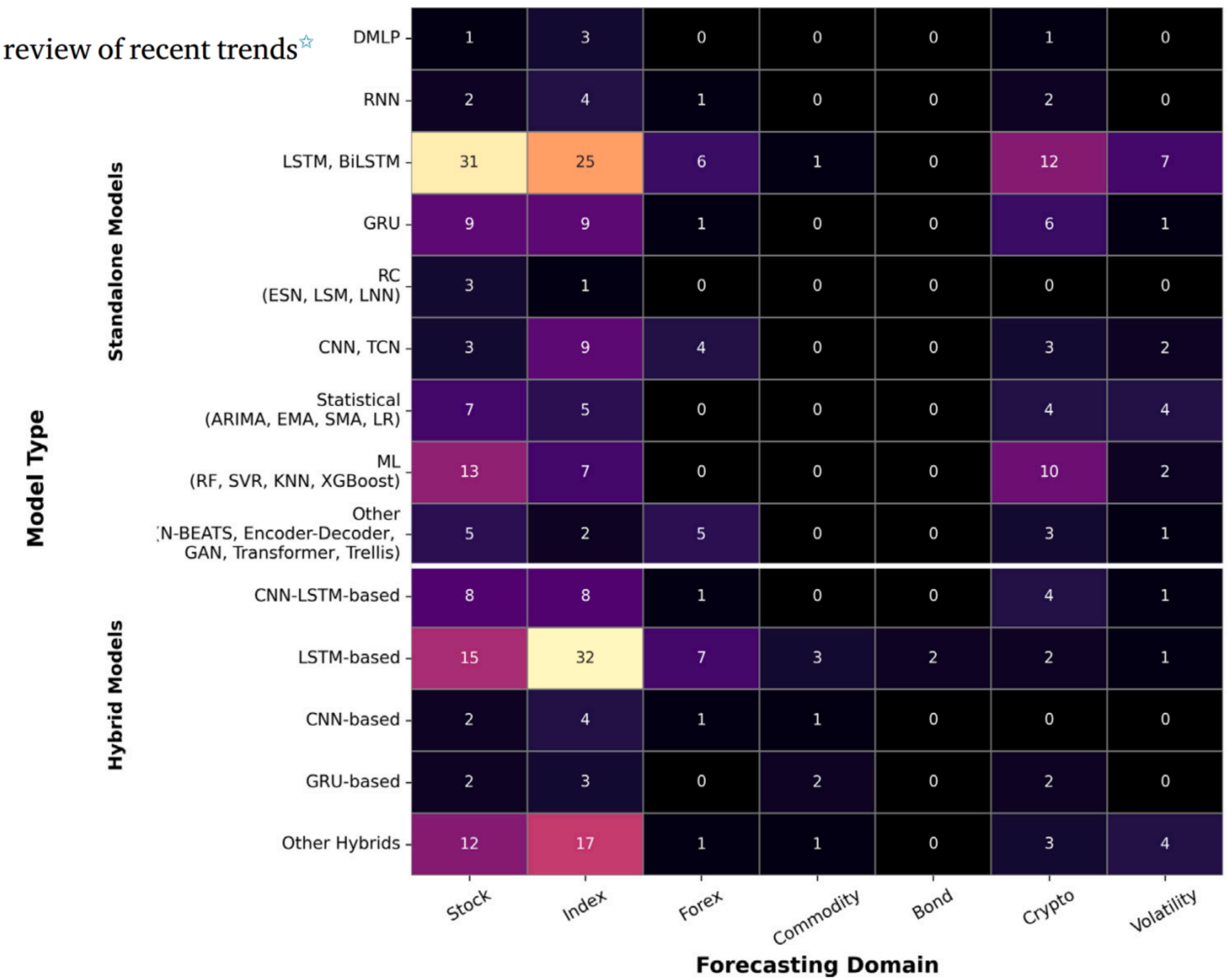
International Review of Economics and Finance

journal homepage: www.elsevier.com/locate/iref

ELSEVIER

Deep learning for financial forecasting: A review of recent trends☆

Sofia Giantsidi [a],*, Claudia Tarantola [b]



Break Down Stock Price Forecasting

Category	Top Entities/Statistics
Most Common Models	Standalone: LSTM (31), ML (13), GRU (9); Hybrids: LSTM-based (15), CNN-LSTM-based (8). Total applications: 117[a b].
Standalone vs. Hybrid Use	Standalone: ~98%; Hybrid/ensemble models: ~52%; Novel architectures: ~21%.[c]
Frequent Feature Types	Nearly all studies (~99%) use price-based features (OHLCV, Close, Adjusted Close); ~23% incorporate technical indicators; ~13% include sentiment (e.g., news, tweets). Notably, ~13% (10 studies) rely solely on Close price data. Rare features include stock bar chart images, financial factors, and correlation data.
Most Common Assets Forecasted	US stocks (e.g., Apple, Tesla, Amazon), Chinese A-shares, Indian stocks (e.g., Axis Bank), Indonesian equities (e.g., PT Bank Central Asia).
Geographic Focus	USA (NASDAQ, NYSE, S&P500, etc.): ~47% (35 studies); China (SSE, SZSE, CSI300): ~15% (11 studies); India (NSE, BSE): ~8% (6 studies); Indonesia: ~3% (2 studies); Multi-region/global: ~13% (10 studies).
Time Periods Covered	Most studies span 2000–2024, with a focus on 2010–2022. Long-term periods (>10 years) are used in ~45% (34 studies), while ~21% (16 studies) examine short periods (<5 years). Some datasets go back to the 1960s.
Sentiment-Integrated Models	10 studies combine market data with sentiment (from Twitter, news, forums). LSTM variants are most used (e.g., ABC-LSTM, Attention-LSTM), along with hybrid and embedding-based models (e.g., User2Vec).

- Other categories:
  - Models trained for individual stocks or an entire industry (pharma, oil, …)

Break Down Index Forecasting

Category	Top Entities/Statistics
Most Common Models	Standalone: LSTM (25), CNN (9), GRU (9); Hybrids: LSTM-based (32), CNN-LSTM-based (8). Total applications: 131[a b].
Standalone vs. Hybrid Use	Standalone: ~78%; Hybrid/ensemble models: ~77%; Novel architectures: ~31%.[c]
Frequent Feature Types	Price-based features (e.g., OHLCV, Close, Adjusted Close) appear in most studies ($\approx 92\%$); technical indicators appear in $\approx 22\%$ of cases; sentiment features (e.g., news, tweets) in only $\approx 6\%$. Notably, $\approx 28\%$ (23 studies) rely solely on Close price data. Rarely used features: chaotic-modeled time series, dividend yields, GAF images.
Most Forecasted Stock Indices	U.S. indices dominate, led by S&P500 (~37%), DJI (~18%), and NASDAQ (~10%). Chinese indices follow with SSE (~19%) and CSI300 (~11%), along with HSI from Hong Kong (~8%) and Nifty50 from India (~7%).
Geographic Focus	Global or mixed-region studies dominate (~40%, 33 studies), followed by the single-study cases U.S. (~22%, 18 studies) and China (~18%, 15 studies). Other regional focuses include India (~6%, 5 studies), Europe (~4%, 3 studies), and Pakistan, Saudi Arabia, South Korea, and Thailand (each ~1%).
Time Periods Covered	Most studies use long-term datasets over 10 years (~70%, 58 studies), while fewer focus on periods <5 years (~28%, 23 studies). The earliest dataset begins in 1983, with many extending into the early 2020s. The 2010–2021 decade is the most studied (~20%, 17 studies). Some analyze multiple periods, but overall, longer historical datasets are preferred.

Hochschule für Technik und Wirtschaft Berlin
University of Applied Sciences

Break Down Crypto

Category	Top Entities/Statistics
Most Common Models	Standalone: LSTM (12), ML (10), GRU (6); Hybrids: CNN-LSTM-based (4). Total applications: 53[a b].
Standalone vs. Hybrid Use	Standalone: ~178%; Hybrid/ensemble models: ~48%; Novel architectures: ~26%.[c]
Frequent Feature Types	Price-based features (e.g., OHLCV, Close, Adjusted Close) are used in all studies (≈ 113%), with some relying exclusively on OHLCV and others combining it with Adjusted Close. Less common features include on-chain data metrics, hash rate[d], and volatility.
Most Forecasted Crypto Assets	Bitcoin dominates (~78%, 18 studies), followed by Ethereum (~39%, 9) and Ripple (~17%, 4).
Time Periods Covered	Nearly half of the studies (~48%, 11 studies) use short-term datasets under five years, while only a few (~13%, 3) span ten years or more, with the longest covering 11 years. Most focus on the post-2015 period, reflecting the maturing phase of cryptocurrency markets.

Trading Template

- https://github.com/2026-03-11-USW/09_Trading

Hochschule für Technik und Wirtschaft Berlin
University of Applied Sciences

Schedule

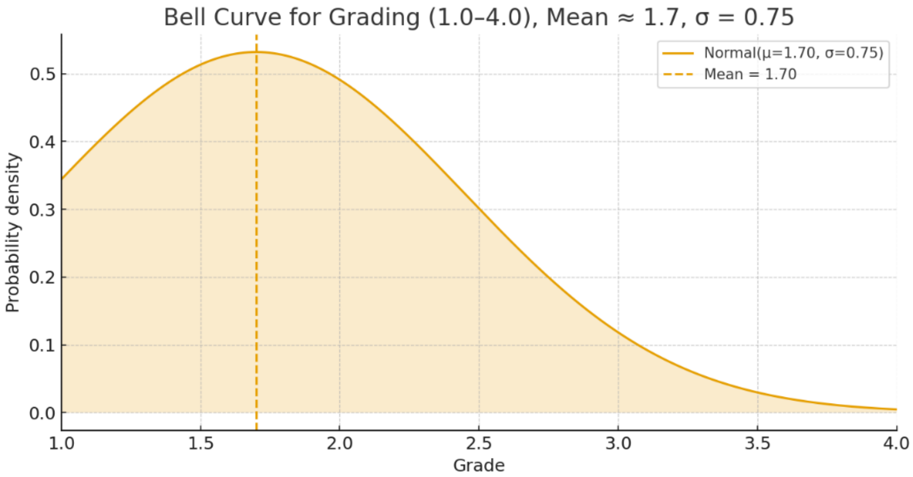
1. 14.04.: Introduction, start hackathon
2. 21.04.: ML basics, data science exercise (assessment)
3. 28.04.: Feed forward neural networks (assessment)
4. 05.05.: RNN, LSTM, CNN (assessment)
5. 12.05.: Transformer architecture, Gen AI (assessment)
6. 19.05.: Gen AI, start projects (assessment)
7. 26.05.: Review
8. 02.06.: Review (Graded)
9. 09.06.: Review
10. 16.06.: Intermediate presentations (Graded)
11. 23.06.: Review
12. 30.06.: Review (Graded)
13. 07.07.: Review
14. 14.07.: Final presentations (Graded)

Review Sessions

- ~20 min. slots per team
- ~10 min. presentation
- Discuss technical & organization issues, questions, feedback, expectations etc.
- Email any time: axel.hochstein@htw-berlin.de
- Additional meetings possible

Grading

- Assessments (individually): 20%
  - During classes each student performs a quick 3-minute assessment covering everything from last weeks class
  - 4 out of 5 assessments count (each 5%)
  - One joker, if all 5 have been submitted the best 4 count
- Project (teams of 3 students): 80%
  - 4 presentations (each 20%)
  - Criteria per presentation (data science)
    - Consideration of specific criteria per DS step (40%)
    - Understanding of code (pseudocode, flowcharts, etc.) (30%)
    - Form of presentation (30%)
      - Freely
      - Clearly, convincing and structured
      - Use of illustrations
  - Criteria per presentation (GenAI)
    - Degree of innovation (20%)
    - Progress (30%)
    - Understanding of code (pseudocode, flowcharts, etc.) (20%)
    - Form of presentation (30%)
      - Freely
      - Clearly, convincing and structured
      - Use of illustrations/prototypes
  - Pick group in Moodle
  - Early communication in case of unbalanced workloads



Hochschule für Technik und Wirtschaft Berlin

University of Applied Sciences

Presentation Problem Definition And Data Acquisition

- **Problem Definition**
  - Describe problem in a concise way
    - What is the target variable?
    - What are input variables?

- **Data Acquisition**
  - What was your approach to acquire required raw data
    - Which APIs did you use?
    - Which parameters did you specify?
    - How did you store your data?
  - Be ready to go through your code for data acquisition
  - Show samples of raw data

Hochschule für Technik
und Wirtschaft Berlin
University of Applied Sciences

Presentation Data Understanding And Data Preparation

- **Data understanding**
  - Explain **relevant** data columns
  - Show **relevant** descriptive statistics of variables
  - Show **relevant** plots of variables
  - Present findings

- **Data preparation (pre-split)**
  - How did you prepare features and targets?
    - Clearly describe the process.
    - Which parameters did you use?
  - Be ready to go through your code for data preparation
  - Show samples of designed features and targets
  - Show **relevant** descriptive statistics for features and targets
  - Show **relevant** plots for features and targets
  - Present findings
  - Explain feature selection process and intuition behind selected features

Presentation Data Preparation, Feature Selection and Modeling

- **Data preparation (post-split) and feature selection**
  - How did you prepare features and targets?
    - Clearly describe the process.
    - Which parameters did you use?
  - Be ready to go through your code for data preparation
  - Show samples of designed features and targets

- **Modeling**
  - Explain in detail the model(s)/algorithm(s) you used for modeling
  - If applicable show network architecture(s)
  - Which parameters did you try?
  - What is your baseline?
  - What are results (train and validation)?

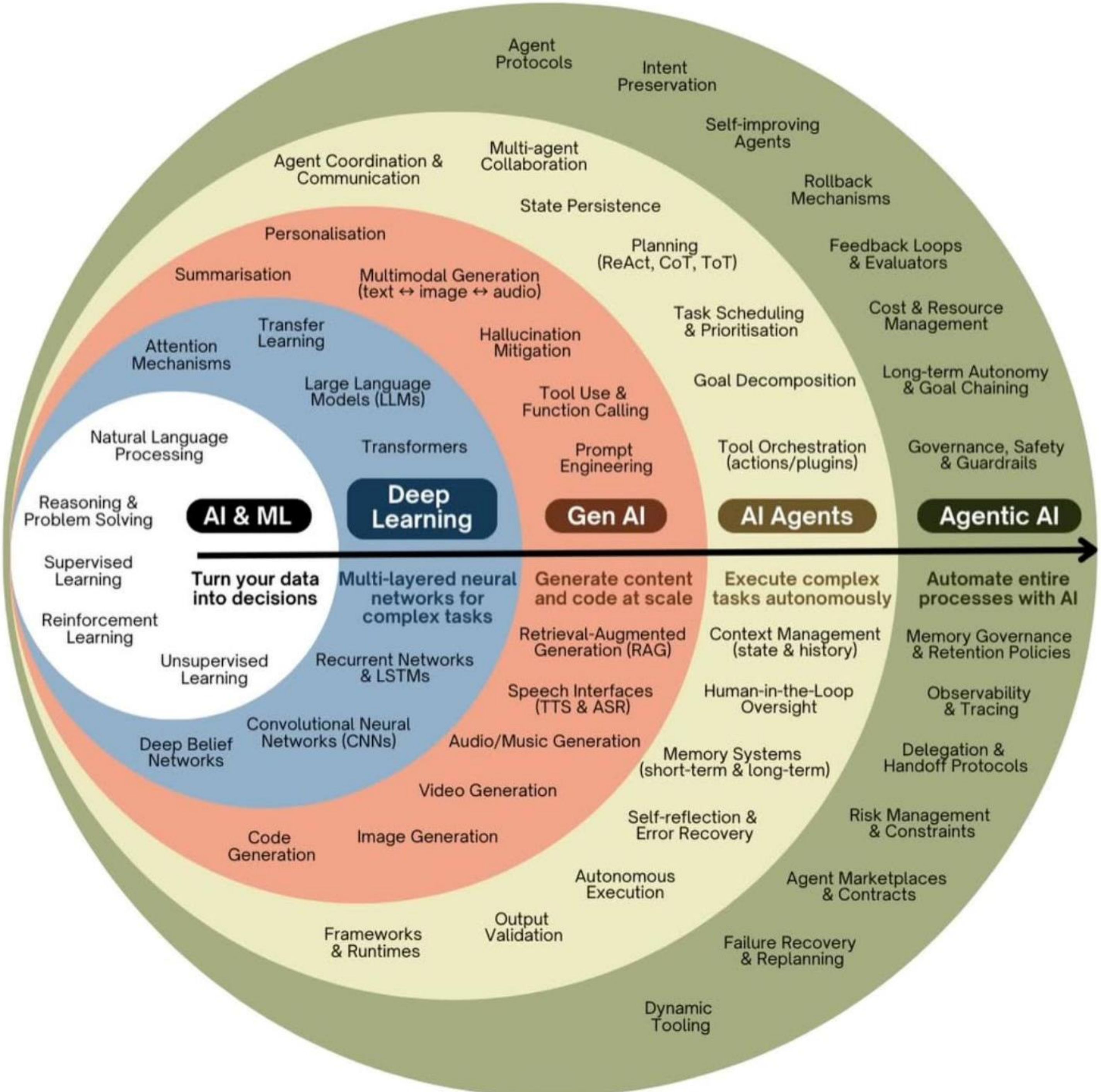
Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Presentation Deployment

- **Deployment**
  - Backtesting trading algorithms
    - How did you derive trading algorithms from trained model?
      - How to specify entry and exit points?
    - How would you have performed in the past?
      - Overall performance
      - Plots of examples
      - Distribution of trading points over time?
      - How did markets develop over time in comparison?
  - Paper trading
    - How did you set up paper trading?
    - Detailed analysis how paper trading algorithms perform?
      - What is the time frame?
      - How did it perform within time frames?
      - How did it perform on an individual basis (e.g. if several stocks are traded)?
      - Does it compare to your backtest results?

- **Next steps**
  - What are your plans to improve results?


AI & ML
Turn your data into decisions
Natural Language Processing
Reasoning & Problem Solving
Supervised Learning
Reinforcement Learning
Unsupervised Learning
Deep Learning
Multi-layered neural networks for complex tasks
Attention Mechanisms
Transfer Learning
Large Language Models (LLMs)
Transformers
Recurrent Networks & LSTMs
Convolutional Neural Networks (CNNs)
Deep Belief Networks
Gen AI
Generate content and code at scale
Personalisation
Summarisation
Multimodal Generation (text ↔ image ↔ audio)
Hallucination Mitigation
Tool Use & Function Calling
Prompt Engineering
Retrieval-Augmented Generation (RAG)
Speech Interfaces (TTS & ASR)
Audio/Music Generation
Video Generation
Image Generation
Code Generation
AI Agents
Execute complex tasks autonomously
Agent Coordination & Communication
Multi-agent Collaboration
State Persistence
Planning (ReAct, CoT, ToT)
Task Scheduling & Prioritisation
Goal Decomposition
Tool Orchestration (actions/plugins)
Context Management (state & history)
Human-in-the-Loop Oversight
Memory Systems (short-term & long-term)
Self-reflection & Error Recovery
Autonomous Execution
Output Validation
Frameworks & Runtimes
Agentic AI
Automate entire processes with AI
Agent Protocols
Intent Preservation
Self-improving Agents
Rollback Mechanisms
Feedback Loops & Evaluators
Cost & Resource Management
Long-term Autonomy & Goal Chaining
Governance, Safety & Guardrails
Memory Governance & Retention Policies
Observability & Tracing
Delegation & Handoff Protocols
Risk Management & Constraints
Agent Marketplaces & Contracts
Failure Recovery & Replanning
Dynamic Tooling

Vibe Coding

- Make sure you understand your code
- Use something like this in your prompt:
  - „Write simple Python code that a beginner can easily understand. Use basic Python features, minimal exception handling, very little abstraction, no unnecessary classes, and no overengineering. Keep it short, readable, and easy to modify.“